

Software Requirements Document

Project Name: LearnGrid LMS

Document Name: LMSAppSRD

Version: 1.0

Date: 04 June 2026

1. Purpose

This Software Requirements Document defines the functional, non-functional, architectural, development, testing, and deployment requirements for LearnGrid LMS, a scalable Learning Management System built using Django, React, PostgreSQL, Kafka, Redis, Selenium, and Tailwind CSS. The system is designed to support approximately 10,000 simultaneous users with reliable performance, modularity, and production-grade deployment practices.

2. Product Overview

LearnGrid LMS is a web-based learning management platform for institutions, instructors, administrators, and students. The platform allows administrators to manage users and courses, instructors to create and deliver learning content, and students to enroll in courses, consume learning materials, complete assessments, track progress, and receive grades and notifications.

3. Technology Stack

Frontend: React, TypeScript, Tailwind CSS, React Router, TanStack Query, Axios, React Hook Form, Zod.

Backend: Django, Django REST Framework, Django settings split by environment, Gunicorn or Uvicorn depending on synchronous/asynchronous service needs.

Database: PostgreSQL with service-level database ownership, PgBouncer connection pooling, backups, indexing, and optional read replicas.

Cache and Temporary Data: Redis for API caching, session/token cache, rate limiting, user permission cache, OTP cache, WebSocket channel layer, and distributed locks.

Event Streaming: Kafka for asynchronous workflows, notifications, progress tracking, grading events, audit logs, and analytics events.

Testing: Pytest, Django Test Framework, DRF API tests, Selenium WebDriver, Selenium Grid, contract tests, load testing with k6 or Locust.

Deployment: Docker, Kubernetes, Helm, Nginx Ingress or API Gateway, CI/CD pipeline, Prometheus, Grafana, Loki or ELK, Sentry, and object storage such as MinIO or S3.

4. User Roles

The system shall support the following primary roles:

Super Admin: Manages platform-wide configuration, institutions, global settings, and system-level monitoring.

Institution Admin: Manages users, courses, batches, departments, instructors, and student enrollments within an institution.

Instructor: Creates courses, uploads content, creates assessments, grades submissions, and views learner progress.

Teaching Assistant: Assists instructors in grading, content management, discussion moderation, and learner support.

Student: Enrolls in courses, consumes lessons, attempts quizzes, submits assignments, tracks progress, and views grades.

Parent or Guardian: Optional role for viewing limited student progress and notifications.

5. Functional Requirements

5.1 Authentication and Authorization

The system shall allow users to register, log in, log out, and reset passwords.

The system shall support JWT-based authentication using access and refresh tokens.

The system shall support role-based access control for Super Admin, Institution Admin, Instructor, Teaching Assistant, Student, and optional Parent/Guardian users.

The system shall enforce backend authorization for every protected API endpoint.

The system shall support object-level authorization so that users can access only permitted courses, enrollments, assessments, submissions, and records.

The system shall support rate limiting for login, OTP, and password reset endpoints using Redis.

5.2 User and Profile Management

The system shall allow administrators to create, update, deactivate, and search users.

The system shall maintain user profile data separately from authentication credentials.

The system shall support student, instructor, admin, and organization-specific profile attributes.

The system shall allow bulk import of users through CSV or Excel in a future release.

5.3 Course Management

The system shall allow instructors and admins to create, update, publish, archive, and delete courses.

The system shall support course categories, tags, difficulty levels, prerequisites, descriptions, thumbnails, and learning outcomes.

The system shall organize courses into modules, lessons, topics, activities, and assessments.

The system shall support draft and published states for courses and lessons.

The system shall support course versioning or content revision history in future releases.

5.4 Content Management

The system shall allow instructors to upload and manage videos, PDFs, documents, images, links, and assignment resources.

The system shall store content metadata in PostgreSQL and actual files in object storage such as MinIO or S3.

The system shall support secure file access using signed URLs or authenticated download endpoints.

The system shall validate file type, file size, and user permissions during upload.

The system shall optionally support video transcoding and adaptive streaming in future releases.

5.5 Enrollment Management

The system shall allow admins or instructors to enroll students into courses.

The system shall support batch-based, cohort-based, and individual enrollment.

The system shall track enrollment status such as active, completed, expired, cancelled, and suspended.

The system shall publish Kafka events when a student is enrolled, removed, or completes a course.

The system shall prevent unauthorized access to courses by non-enrolled students.

5.6 Learning Progress Tracking

The system shall track lesson views, video progress, quiz attempts, assignment submissions, and course completion percentage.

The system shall update progress asynchronously using Kafka where appropriate.

The system shall provide student dashboards showing active courses, completed lessons, pending assessments, grades, and upcoming deadlines.

The system shall provide instructor dashboards showing learner engagement, progress distribution, and assessment status.

5.7 Assessment Management

The system shall allow instructors to create quizzes, question banks, assignments, and exams.

The system shall support question types including multiple choice, multiple select, true or false, short answer, essay, file upload, and coding question in future releases.

The system shall support quiz attempts, time limits, start/end windows, randomization, and auto-submit.

The system shall store submissions securely and preserve audit trails.

The system shall support automated grading for objective questions and manual grading for subjective submissions.

5.8 Grading and Results

The system shall calculate grades based on configured grading rules.

The system shall allow instructors to review, override, and publish grades.

The system shall notify students when grades are published.

The system shall maintain a grade history and audit trail for changes.

The system shall support course completion certificates in a future release.

5.9 Notifications

The system shall notify users about enrollment, assignment deadlines, grade publication, announcements, course completion, and account-related events.

The system shall support in-app notifications initially.

The system shall support email, SMS, and push notifications in future phases.

The notification service shall consume Kafka events and generate notifications asynchronously.

5.10 Search and Reporting

The system shall support searching courses, users, enrollments, assessments, and submissions.

The system shall provide admin reports for active users, course enrollments, completion rates, assessment results, and system usage.

The system shall avoid heavy analytics queries on transactional databases.

The system shall use a separate analytics database or reporting store for large-scale reports.

6. Non-Functional Requirements

6.1 Performance

The system shall support approximately 10,000 simultaneous users.

Common API requests should target a p95 latency below 300 ms under normal load, excluding large file downloads and video streaming.

The system shall use Redis caching for frequently accessed data such as course catalogs, permissions, dashboards, and public metadata.

All list APIs shall support pagination, filtering, and sorting.

The system shall avoid N+1 query patterns using `select_related`, `prefetch_related`, optimized serializers, and proper indexing.

6.2 Scalability

All Django microservices shall be stateless where possible and horizontally scalable.

The React frontend shall be served through CDN or Nginx with caching and compression.

Kafka topics shall use partitions to support parallel consumers.

PostgreSQL shall use PgBouncer for connection pooling and read replicas for read-heavy workloads where required.

Redis shall be deployed using Redis Sentinel or Redis Cluster for production reliability.

6.3 Reliability and Availability

The production system shall be deployed on Kubernetes with multiple replicas for critical services.

The system shall include readiness and liveness probes for all services.

The system shall use retries, dead-letter topics, and idempotent event consumers for Kafka-based workflows.

The system shall support scheduled PostgreSQL backups and point-in-time recovery.

The system shall maintain separate staging and production environments.

6.4 Security

All external traffic shall use HTTPS.

Secrets shall be stored in Kubernetes Secrets, sealed secrets, or Vault, not in source code.

The system shall validate all inputs using serializers, schema validation, and frontend form validation.

The system shall enforce CORS restrictions.

The system shall protect sensitive endpoints using authentication, authorization, rate limiting, and audit logs.

Uploaded files shall be validated for type and size and optionally scanned for malware. The system shall maintain audit logs for login attempts, permission changes, enrollment changes, grade changes, and administrative actions.

6.5 Maintainability

Each service shall follow clean architecture principles with separation between API layer, validation layer, business logic, selectors, models, permissions, and tests.

Each service shall expose OpenAPI documentation.

Each service shall include unit tests, API tests, integration tests, and service-level documentation.

The codebase shall follow formatting and linting standards.

7. Microservice Architecture

The system shall be divided into independently deployable services. Each service owns its data and exposes APIs for other services or frontend clients.

7.1 Auth Service

Responsibilities: login, logout, token generation, refresh tokens, password reset, user credential management, authentication audit logs, token blacklist, and rate limiting.

Primary database: auth_db.

Redis usage: token blacklist, login throttling, password reset OTP cache.

7.2 User/Profile Service

Responsibilities: user profile data, student profile, instructor profile, admin profile, institution profile, department details, and profile search.

Primary database: user_db.

7.3 Course Service

Responsibilities: course catalog, course metadata, modules, lessons, topics, categories, tags, course status, and learning outcomes.

Primary database: course_db.

Redis usage: course catalog cache and frequently accessed metadata.

Kafka events: CourseCreated, CoursePublished, CourseArchived, LessonPublished.

7.4 Content Service

Responsibilities: content metadata, file uploads, file permissions, content version metadata, signed access URLs, and object storage integration.

Primary database: content_db.

Storage: MinIO or S3.

Kafka events: ContentUploaded, ContentPublished, ContentDeleted.

7.5 Enrollment Service

Responsibilities: enrollment, batch assignment, cohort assignment, course access, enrollment status, and enrollment history.

Primary database: enrollment_db.

Kafka events: StudentEnrolled, StudentRemovedFromCourse, CourseAccessExpired.

7.6 Progress Service

Responsibilities: lesson progress, video progress, quiz progress, course progress, completion percentage, and learning analytics events.

Primary database: progress_db.

Kafka events consumed: LessonViewed, VideoCompleted, QuizSubmitted, AssignmentSubmitted.

Kafka events produced: CourseProgressUpdated, CourseCompleted.

7.7 Assessment Service

Responsibilities: quizzes, question banks, assignments, attempts, submissions, answer records, and assessment configuration.

Primary database: assessment_db.

Kafka events: QuizSubmitted, AssignmentSubmitted, AssessmentWindowClosed.

7.8 Grading Service

Responsibilities: automatic grading, manual grading workflow, grade calculation, grade publishing, grade audit history, and result release.

Primary database: grading_db.

Kafka events consumed: QuizSubmitted, AssignmentSubmitted.

Kafka events produced: GradeCalculated, GradePublished, CertificateEligible.

7.9 Notification Service

Responsibilities: in-app notifications, email notifications, SMS integration, push notifications, notification templates, and delivery status.

Primary database: notification_db.

Kafka events consumed: StudentEnrolled, AssignmentDueSoon, GradePublished, CourseCompleted.

7.10 Analytics Service

Responsibilities: dashboards, reporting, engagement metrics, usage analytics, course performance, student performance, and institutional reports.

Primary database: analytics_db or analytical store.

Kafka events consumed: user.events, course.events, enrollment.events, progress.events, assessment.events, grading.events.

8. Database Architecture

The preferred production model is database-per-service. Each service shall own its database schema and shall not directly query another service database.

Recommended databases:

auth_db, user_db, course_db, content_db, enrollment_db, progress_db, assessment_db, grading_db, notification_db, analytics_db.

Database rules:

Use migrations for all schema changes.

Use indexes for foreign keys, lookup fields, status fields, and frequently filtered fields.

Use pagination for all list endpoints.

Use soft delete where business history must be preserved.

Use UUIDs for public identifiers where appropriate.

Use database constraints for critical integrity rules.

Use transactions for multi-step writes inside a service boundary.

Avoid cross-service database joins.

9. Kafka Architecture

Kafka shall be used for asynchronous, event-driven workflows.

Recommended Kafka topics:

user.events

course.events

enrollment.events

content.events

progress.events

assessment.events

grading.events

notification.events

audit.events

analytics.events

Kafka design rules:

Each event shall include event_id, event_type, aggregate_id, producer_service, timestamp, version, correlation_id, and payload.

Consumers shall be idempotent.

Failed events shall be retried using retry topics.

Poison messages shall be moved to dead-letter topics.

Consumer lag shall be monitored.

Kafka should not be used for synchronous request-response queries.

Example event:

```
{
  "event_id": "uuid",
  "event_type": "StudentEnrolled",
  "student_id": "student_uuid",
  "course_id": "course_uuid",
  "timestamp": "2026-06-04T10:00:00Z",
  "producer_service": "enrollment-service"
}
```

10. Redis Architecture

Redis shall be used for temporary, high-speed data access.

Redis use cases:

API response cache

Course catalog cache

User permission cache

JWT blacklist

OTP and password reset token cache

Rate limiting

WebSocket channel layer

Distributed locks

Temporary assessment attempt state

Redis rules:

Do not store permanent academic records only in Redis.

Use TTLs for cached data.

Invalidate cache after writes where stale data is harmful.

Separate Redis workloads if required, for example cache Redis, session Redis, and channel Redis.

11. API Gateway Requirements

The system shall use an API Gateway such as Kong, Nginx, Traefik, or equivalent.

Gateway responsibilities:

TLS termination

Routing to microservices

Rate limiting

Request logging

CORS control

Authentication middleware where applicable

API versioning

Request size limits

Example routes:

/api/auth/* -> auth-service

/api/users/* -> user-service

/api/courses/* -> course-service

/api/content/* -> content-service

/api/enrollments/* -> enrollment-service

/api/progress/* -> progress-service

/api/assessments/* -> assessment-service

/api/grades/* -> grading-service

/api/notifications/* -> notification-service

/api/analytics/* -> analytics-service

12. Frontend Requirements

The frontend shall be developed using React, TypeScript, and Tailwind CSS.

The frontend shall use reusable components, route-based code splitting, protected routes, and role-based UI rendering.

The frontend shall use TanStack Query or equivalent for server-state caching.

The frontend shall use React Hook Form and Zod for form handling and validation.

The frontend shall handle loading states, error states, empty states, and retry states.

The frontend shall implement separate layouts for student, instructor, and admin portals.

The frontend shall not be trusted for authorization; all authorization must be enforced by backend services.

Suggested frontend structure:

frontend/src/app

frontend/src/api

frontend/src/components

frontend/src/features/auth

frontend/src/features/courses

frontend/src/features/enrollment

frontend/src/features/assessments

frontend/src/features/dashboard

frontend/src/features/profile

frontend/src/hooks

frontend/src/layouts

frontend/src/routes

frontend/src/stores

frontend/src/types

frontend/src/utils

13. Backend Development Standards

Each Django service shall use Django REST Framework.

Each service shall follow this internal structure:

config

apps/<domain>/models.py

apps/<domain>/serializers.py

apps/<domain>/views.py

apps/<domain>/services.py

apps/<domain>/selectors.py

apps/<domain>/permissions.py

apps/<domain>/urls.py

apps/<domain>/tests

Backend design rules:

views.py handles HTTP request and response logic.

serializers.py handles validation and representation.

services.py contains business workflows and write operations.
selectors.py contains optimized read queries.
permissions.py contains authorization rules.
models.py contains database schema and model constraints.

14. Testing Requirements

14.1 Unit Testing

Each service shall include unit tests for models, services, selectors, permissions, and serializers.

Business logic shall be tested independently from API views where possible.

14.2 API Testing

Each service shall include API tests for authentication, permissions, validation, happy paths, and failure paths.

14.3 Integration Testing

Integration tests shall verify database operations, Redis caching, Kafka producers, Kafka consumers, and object storage operations.

Testcontainers may be used to run PostgreSQL, Redis, and Kafka in CI.

14.4 Selenium End-to-End Testing

Selenium shall be used for browser-based end-to-end testing.

Selenium tests shall cover:

Student login

Instructor login

Admin login

Course creation

Course publishing

Student enrollment

Lesson viewing

Quiz attempt

Assignment submission

Grade viewing

Role-based access control

Logout

Recommended Selenium structure:

tests/e2e/pages/login_page.py

tests/e2e/pages/dashboard_page.py

tests/e2e/pages/course_page.py

tests/e2e/pages/assessment_page.py

tests/e2e/tests/test_login.py

tests/e2e/tests/test_course_enrollment.py

tests/e2e/tests/test_lesson_completion.py
tests/e2e/tests/test_quiz_submission.py

14.5 Load Testing

The system shall be load tested before production release.

Load testing shall simulate login, dashboard loading, course listing, lesson access, quiz submission, and notification flows.

Tools: k6, Locust, or JMeter.

Key metrics: p95 latency, error rate, throughput, PostgreSQL connections, Redis memory, Kafka lag, CPU, memory, and pod autoscaling behavior.

15. Deployment Architecture

The production system shall be deployed using Docker containers and Kubernetes.

Each microservice shall have its own Dockerfile, Kubernetes Deployment, Service, ConfigMap, Secret, and Horizontal Pod Autoscaler.

The frontend shall be built as a static React application and served using Nginx or CDN.

The API Gateway shall route requests to backend services.

PostgreSQL, Redis, Kafka, and MinIO/S3 shall be deployed as managed services or production-grade clustered services.

Kubernetes components:

ingress-nginx or API gateway

frontend deployment

auth-service deployment

user-service deployment

course-service deployment

content-service deployment

enrollment-service deployment

progress-service deployment

assessment-service deployment

grading-service deployment

notification-service deployment

analytics-service deployment

Redis cluster or Sentinel

Kafka cluster

PostgreSQL primary and replicas

MinIO or S3 integration

Prometheus

Grafana

Loki or ELK

Sentry

16. CI/CD Requirements

The CI/CD pipeline shall perform:

- Code checkout
- Static analysis
- Linting
- Type checking
- Unit tests
- API tests
- Docker image build
- Security scan
- Push image to registry
- Deploy to staging
- Run integration tests
- Run Selenium smoke tests
- Run load test subset when required
- Manual approval for production
- Production deployment
- Post-deployment smoke tests

17. Observability Requirements

The system shall include centralized metrics, logs, traces, and error reporting.

Required tools:

- Prometheus for metrics
- Grafana for dashboards
- Loki or ELK for logs
- Jaeger or Tempo for distributed tracing
- Sentry for application errors
- Kafka UI for Kafka monitoring
- PostgreSQL exporter
- Redis exporter

Important dashboards:

- API latency dashboard
- Error rate dashboard
- Kafka consumer lag dashboard
- PostgreSQL health dashboard
- Redis memory and hit-rate dashboard
- Kubernetes pod health dashboard
- User activity dashboard
- Assessment submission dashboard

18. Security Requirements

The system shall implement:

- HTTPS everywhere
- JWT access and refresh tokens
- RBAC and object-level permission checks

Rate limiting for sensitive APIs
Audit logging
Input validation
File upload validation
Secure password hashing
Secrets management
CORS restrictions
CSRF protection where cookie-based auth is used
Secure headers
Database backups and restore testing
Least-privilege service accounts

19. Initial Development Roadmap

Phase 1: Foundation

Set up monorepo, React frontend, Tailwind CSS, Auth Service, User Service, Course Service, PostgreSQL, Redis, Docker Compose, and basic CI.

Phase 2: LMS Core

Implement course creation, modules, lessons, student enrollment, student dashboard, instructor dashboard, admin dashboard, and role-based permissions.

Phase 3: Assessment

Implement quiz creation, question banks, attempts, assignment submission, automatic grading, manual grading, and grade publication.

Phase 4: Event-Driven Architecture

Introduce Kafka, publish enrollment events, publish progress events, publish grading events, build notification consumers, and build analytics consumers.

Phase 5: Testing and Quality

Add unit tests, API tests, integration tests, contract tests, Selenium E2E tests, load tests, and security checks.

Phase 6: Production Deployment

Deploy to Kubernetes using Helm, configure monitoring, logging, backups, autoscaling, staging environment, production environment, and disaster recovery procedures.

20. Acceptance Criteria

The application shall allow users to authenticate and access role-specific dashboards.

Admins shall be able to manage users, instructors, students, and courses.

Instructors shall be able to create courses, upload content, create assessments, and publish grades.

Students shall be able to enroll in courses, view lessons, complete assessments, and track progress.

Kafka-based events shall be generated for enrollment, progress, assessment, grading, notification, audit, and analytics workflows.

Redis shall be used for caching, rate limiting, token blacklist, and temporary workflow data.

PostgreSQL shall store transactional data with proper migrations, indexing, constraints, and backups.

Selenium E2E tests shall validate the main user journeys.

The system shall be deployable to Kubernetes using Docker images and CI/CD.

The system shall be observable using metrics, logs, traces, and alerts.

The system shall be tested for high concurrency before production release.

21. Recommended First Production Scope

The first production version should include the following services:

auth-service

user-service

course-service

enrollment-service

assessment-service

notification-service

The following services can be added in later phases:

progress-service

grading-service

analytics-service

content-service enhancements

certificate-service

payment-service if commercial course sales are required

22. Open Decisions

Select final API Gateway: Kong, Traefik, or Nginx.

Select object storage: MinIO for on-premise or S3-compatible cloud storage.

Select deployment model: fully on-premise Kubernetes, cloud Kubernetes, or hybrid.

Finalize authentication model: JWT only, OAuth2, or SSO integration.

Finalize analytics storage: PostgreSQL reporting database, ClickHouse, or data warehouse.

Finalize video delivery strategy: direct object storage, CDN, or streaming server.

23. Revision History

Version 1.0: Initial SRD created for LearnGrid LMS using Django, React, PostgreSQL, Kafka, Redis, Selenium, Tailwind CSS, Docker, and Kubernetes.